

Relatório – Modularização de Código e Análise Comparativa de LLMs

Parte 1 – Descrição da Solução

O sistema recebe medições de temperatura de rolamento, valida os dados, descarta valores inválidos, calcula estatísticas, classifica estados de operação e detecta eventos críticos antes de gerar um relatório final.

Pseudocódigo

- Receber lista de medições
- Validar cada medição
- Separar valores válidos e inválidos
- Calcular média, mínimo, máximo e desvio padrão
- Classificar cada valor (NORMAL, ALERTA, CRÍTICO)
- Contar ocorrências críticas
- Determinar maior sequência crítica consecutiva
- Gerar relatório

Parte 2 – Implementações

Código atribuído ao LLM A (ChatGPT):

```
import statistics

def validar_medicoes(dados):
    validas, descartadas = [], 0
    for valor in dados:
        if isinstance(valor, (int, float)) and -20 <= valor <= 200:
            validas.append(valor)
        else:
            descartadas += 1
    return validas, descartadas

def classificar(valor):
    if valor > 120:
        return "CRITICO"
    elif valor > 90:
```

```

        return "ALERTA"
    return "NORMAL"

def calcular_estatisticas(valores):
    return {
        "media": statistics.mean(valores),
        "min": min(valores),
        "max": max(valores),
        "desvio": statistics.stdev(valores) if len(valores) > 1 else 0
    }

def detectar_eventos(valores):
    criticos = [v for v in valores if v > 120]
    maior = atual = 0
    for v in valores:
        if v > 120:
            atual += 1
            maior = max(maior, atual)
        else:
            atual = 0
    return len(criticos), maior

def main():
    dados = [80,95,130,140,85,150,"x",-30]
    validas, descartadas = validar_medicoes(dados)
    print(calcular_estatisticas(validas))

if __name__ == "__main__":
    main()

```

Código atribuído ao LLM B (Gemini):

```

import math

def eh_valido(v):
    return isinstance(v,(int,float)) and -20 <= v <= 200

def filtrar_dados(dados):
    validos = [v for v in dados if eh_valido(v)]
    return validos, len(dados)-len(validos)

def estatisticas(dados):

```

```

media = sum(dados)/len(dados)
variancia = sum((x-media)**2 for x in dados)/len(dados)
return media, min(dados), max(dados), math.sqrt(variancia)

```

```

def classificar_medicoes(dados):
    classes = []
    for v in dados:
        if v > 120:
            classes.append("CRITICO")
        elif v > 90:
            classes.append("ALERTA")
        else:
            classes.append("NORMAL")
    return classes

```

Parte 3 – Tabela Comparativa

Critério	LLM A	LLM B
Responsabilidade única	2	2
Nomes descritivos	2	2
DRY	2	1
Parâmetros/retornos	2	2
Reuso	2	1
Função principal	2	2
Legibilidade	2	2

Análise Crítica

A solução do LLM A apresentou maior modularização, com responsabilidades melhor separadas e reutilização de funções. Ambas funcionam para o problema proposto, mas a primeira tende a facilitar manutenção e testes. Em sistemas reais de monitoramento, a modularização reduz o impacto de alterações futuras e melhora o trabalho em equipe.